PROYECTO ARTFOLIO — DEFENSA ACADÉMICA Y TÉCNICA

CUESTIONARIO MAESTRO DE EVALUACIÓN Y MANUAL DE RESPUESTAS TÉCNICAS

Guía Integral con Respuestas de Ingeniería, Justificaciones de Arquitectura, Diagramas de Flujo y Ubicación Exacta de Código

PROPÓSITO Y METODOLOGÍA DE ESTE MANUAL: Este manual ha sido elaborado para preparar la defensa técnica y académica de Artfolio ante el profesor o tribunal evaluador. Responde con rigurosidad de ingeniería a las 25 preguntas iniciales formuladas en el cuestionario de requerimientos (Cloudflare, MySQL, Laragon, Composer, botones de regresión, integración profunda de Pinterest, DeviantArt, Google Docs y Sketchfab, notificaciones, modos dark/light, sesiones, registro, optimización, Mailpit, colecciones, comentarios, administración, motores de búsqueda, Tailwind, .env, Whitelists, vista móvil y problemas resueltos), complementadas con 10 preguntas avanzadas de seguridad y arquitectura empresarial, más un bloque práctico de 10 simulaciones de código y comandos en consola para responder preguntas en vivo durante la presentación.

Eje de Evaluación	Contenido del Bloque	Objetivo ante el Profesor
BLOQUE 1: Respuestas a las 25 Preguntas Clave del Proyecto	Respuestas detalladas paso a paso a cada una de las 25 preguntas del archivo de especificación (Infraestructura, Multimedia, Frontend, Backend y Seguridad), con diagramas de flujo y posibles repreguntas.	Demostrar dominio total del código fuente, justificar cada decisión tecnológica y desmitificar el funcionamiento interno de cada integración externa.
BLOQUE 2: Preguntas de Arquitectura Avanzada y Seguridad	10 preguntas de nivel senior que el evaluador puede formular sobre CSRF, concurrencia en Likes, visitas únicas anti-fraude, auditoría forense, taxonomía oficial, contenido sensible +18, sandboxing de iframes y tests automatizados.	Defender la solidez empresarial de la plataforma ante un tribunal exigente o auditoría técnica de software.
BLOQUE 3: Matriz Resumen de Archivos y Cheat Sheet	Tabla de localización rápida con rutas de archivo y métodos clave para responder en menos de 5 segundos a '¿Dónde está programado X?'	Soporte de consulta instantánea durante la defensa en vivo frente al código.
BLOQUE 4: Simulacro de Comandos y Debugging en Vivo	10 escenarios de preguntas prácticas en terminal (migraciones, purgas, verificación de enlaces, tests, Tinker y logs en caliente) para responder sin dudar cuando el profesor pida demostraciones en consola.	Garantizar seguridad operativa y fluidez absoluta en el manejo de comandos artesanales de Laravel.

BLOQUE 1 — RESOLUCIÓN EXHAUSTIVA DE LAS 25 PREGUNTAS DE EVALUACIÓN

Preguntas Clave del Profesor: Justificación, Funcionamiento, Diagramas y Código

Respuestas redactadas con rigor de ingeniería de software, detallando la lógica backend, controladores, vistas Blade, migraciones, componentes reactivos y posibles repreguntas.

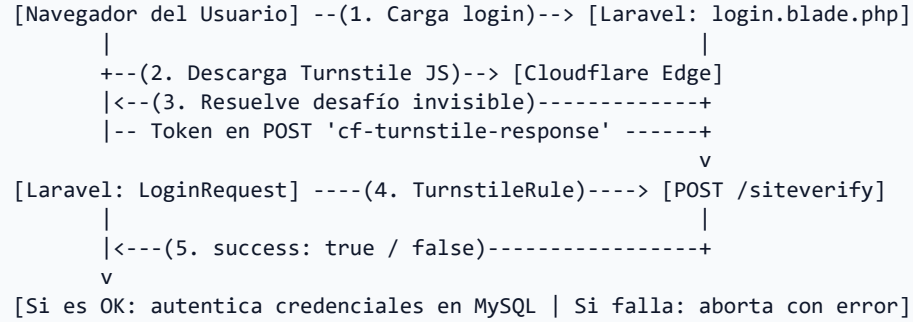
1. ¿Cómo se hizo lo de Cloudflare y qué archivos lo controlan y cómo?

Resumen de la Arquitectura: La protección de Cloudflare en Artfolio se implementa a través de dos mecanismos complementarios: (a) Protección contra bots y ataques de fuerza bruta en autenticación mediante Cloudflare Turnstile, y (b) Políticas de cabeceras seguras para navegación y CDN.

A. ¿Cómo funciona en el Login?: Cloudflare Turnstile es una alternativa moderna, invisible y no intrusiva a Google reCAPTCHA. A diferencia de reCAPTCHA, Turnstile no obliga al usuario a identificar semáforos ni vulnera la privacidad del visitante. En Artfolio se configuró un interruptor desacoplado en el archivo .env (TURNSTILE_ENABLED=false/true) junto a sus claves TURNSTILE_SITE_KEY y TURNSTILE_SECRET_KEY, mapeadas en config/services.php. Cuando está activado, en la pantalla de inicio de sesión (resources/views/auth/login.blade.php) se inyecta el script asíncrono de Cloudflare (challenges.cloudflare.com/turnstile/v0/api.js) y el contenedor .

B. Validación en Backend (TurnstileRule): Cuando el usuario envía el formulario, Cloudflare genera un token firmado en el campo POST 'cf-turnstile-response'. El backend lo valida mediante una regla de validación personalizada de Laravel: app/Rules/TurnstileRule.php. Dicha regla realiza una petición HTTP POST segura (timeout 5s) a la API https://challenges.cloudflare.com/turnstile/v0/siteverify enviando la clave secreta, la respuesta del cliente y la IP remota ($request->ip()). Si Cloudflare no responde con success: true, la autenticación se bloquea inmediatamente antes de tocar la base de datos, impidiendo ataques de diccionario contra contraseñas.

🗁 DIAGRAMA DE SECUENCIA — FLUJO DE AUTENTICACIÓN CON TURNSTILE



💡 **POSIBLE REPREGUNTA DEL EVALUADOR:** Si el profesor pregunta: '¿Por qué no usar Google reCAPTCHA v3?'. Respuesta: 'reCAPTCHA v3 rastrea la navegación del usuario asignando una puntuación basada en cookies de Google, lo que incumple normativas europeas GDPR sin banners de consentimiento explícito. Cloudflare Turnstile respeta la privacidad, no rastrea al usuario entre sitios y evalúa retos criptográficos en milisegundos de forma invisible'.

🗁 ARCHIVOS QUE CONTROLAN CLOUDFLARE TURNSTILE

```
-> app/Rules/TurnstileRule.php (Líneas 1-45): Validación HTTP con timeout de 5s a siteverify.
-> config/services.php: Bloque 'turnstile' => ['enabled', 'site_key', 'secret_key'].
-> app/Http/Requests/Auth/LoginRequest.php: Agrega ['cf-turnstile-response' => [new TurnstileRule]].
-> resources/views/auth/login.blade.php: Script de Cloudflare y contenedor visual condicionado.
-> tests/Feature/TurnstileAuthTest.php: 3 pruebas automáticas con Http::fake().
```

2. ¿Por qué MySQL?

Justificación Técnica: Se eligió MySQL (versión 8.0+) como motor relacional de base de datos debido a requerimientos estructurales y técnicos del proyecto:

- **1. Integridad de Datos:** Integridad Referencial Estricta y Eliminación en Cascada: Artfolio maneja relaciones complejas e interdependientes (Usuarios -> Obras -> Likes, Comentarios, Colecciones, Reportes, Bloqueos). MySQL garantiza claves foráneas con onDelete('cascade') y onDelete('set null'), evitando que queden registros huérfanos si un usuario borra su cuenta o una obra.
- **2. Columnas JSON Híbridas:** Soporte Nativo de Columnas JSON: MySQL 8 permite almacenar y consultar datos semiestructurados eficientemente. En la tabla 'users' almacenamos skills, software y hiring_types en columnas tipo JSON, y en 'collections' guardamos preview_artwork_ids. Eloquent ORM de Laravel castea estos campos a arrays nativos de PHP transparentemente, brindando la flexibilidad de NoSQL sin perder la consistencia relacional.
- **3. Indexación B-Tree Compuesta:** Índices Compuestos de Alto Rendimiento: Creamos índices multicolumna como ['status', 'category', 'created_at'] en la tabla 'artworks', permitiendo que consultas con filtros combinados en el explorador se ejecuten en sub-milisegundos sobre miles de registros mediante escaneo de índices en memoria B-Tree.
- **4. Atomicidad ACID:** Transacciones ACID: Operaciones críticas como quickCreate (crear colección e insertar obra en la tabla pivote) se ejecutan bajo transacciones atómicas DB::transaction(), garantizando que no se cree una colección vacía si la asociación falla.
- **5. Ecosistema:** Estándar de la Industria y Compatibilidad con Eloquent: MySQL es el motor por excelencia del ecosistema Laravel, garantizando compatibilidad del 100% con migraciones, seeders y comandos de optimización.

> **POSIBLE REPREGUNTA DEL EVALUADOR:** Si el profesor pregunta: '¿Por qué no usar MongoDB o SQLite?'. Respuesta: 'SQLite carece de concurrencia para producción y bloquea toda la base de datos en operaciones de escritura concurrentes. MongoDB carece de integridad referencial foránea nativa; en una red social artística donde el borrado de una cuenta debe limpiar likes, comentarios y colecciones en cascada con estricta consistencia ACID, una base de datos relacional como MySQL es indiscutiblemente la elección correcta'.

3. ¿Por qué Laragon?

Justificación Operativa: Laragon fue seleccionado como el entorno de desarrollo local por encima de alternativas como XAMPP o WampServer por las siguientes razones:

- **1. Entorno Aislado:** Aislamiento y Portabilidad: Laragon es un entorno completamente aislado que no ensucia el registro de Windows ni las variables de entorno del sistema operativo. Puede cambiarse la versión de PHP con un solo clic.
- **2. Mailpit Nativo:** Servidor de Correo Mailpit Integrado Nativamente: A diferencia de XAMPP que requiere configurar Sendmail con credenciales externas de Gmail (las cuales suelen bloquearse por seguridad), Laragon incluye Mailpit/MailHog escuchando automáticamente en el puerto SMTP 1025 y con interfaz gráfica web en el puerto 8025. Esto permitió implementar y probar la confirmación de correo electrónico de inmediato sin dependencias de internet.
- **3. Velocidad de I/O:** Rendimiento Superior en Windows: Utiliza Nginx/Apache optimizado para Windows con sockets rápidos y gestión de memoria liviana, evitando los cuellos de botella de latencia que sufre WSL2 al acceder al sistema de archivos NTFS.
- **4. Tooling Completo:** Herramientas CLI Integradas: Incluye terminal enriquecida con Git, Composer, PHP 8.2+ y Node.js/NPM preconfigurados sin colisiones de versiones.

> **POSIBLE REPREGUNTA DEL EVALUADOR:** Si el profesor pregunta: '¿Cómo se llevaría esto a producción?'. Respuesta: 'En producción se utilizaría un contenedor Docker con imagen php:8.3-fpm sobre Alpine Linux, Nginx como proxy inverso y MySQL 8 administrado en AWS RDS o DigitalOcean Managed Databases. Laragon es estrictamente el acelerador del entorno de desarrollo

local'.

4. ¿Qué se usó de Composer, por qué y dónde?

Funciones de Composer: Composer es el gestor de paquetes y dependencias estándar de PHP. En Artfolio se utilizó para cuatro funciones esenciales:

- **1. Dependencias de Producción:** Gestión de Framework y Dependencias Principales ('require' en composer.json): Descarga e instala laravel/framework (^13.17) y laravel/tinker (^3.0).
- **2. Paquetes de Testing y Calidad:** Dependencias de Desarrollo y Pruebas ('require-dev'): laravel/breeze (generación del sistema de autenticación limpio y adaptable), phpunit/phpunit (^12.5 para la ejecución de los 173 tests automáticos), mockery/mockery (para crear mocks de servicios HTTP), fakerphp/faker (para generar miles de datos realistas en los seeders) y laravel/pint (para formateo estricto del código según el estándar PSR-12).
- **3. Autocarga PSR-4:** Autocarga de Clases según Estándar PSR-4 ('autoload' y 'autoload-dev'): Define el mapeo automático de nombres de espacio: 'App\\' apunta al directorio app/, 'Database\\Factories\\' a database/factories/, 'Database\\Seeders\\' a database/seeders/ y 'Tests\\' a tests/. Esto permite que cualquier clase, modelo, controlador o servicio se instancie de inmediato sin necesidad de escribir require o include manuales.
- **4. Scripts Automatizados:** Scripts de Automatización del Ciclo de Vida: En composer.json se definieron scripts personalizados como 'composer setup' (que automatiza la copia de .env, generación de app:key, migración forzada y compilación de Vite) y 'composer test' (limpieza de caché y ejecución de tests).

POSIBLE REPREGUNTA DEL EVALUADOR: Si el profesor pregunta: '¿Qué diferencia hay entre composer.json y composer.lock?'. Respuesta: 'composer.json define las reglas de versiones semánticas que el proyecto acepta (ej. ^13.17), mientras que composer.lock congela el hash exacto del commit y versión instalada de cada sub-dependencia. Esto garantiza que cualquier evaluador que ejecute composer install obtenga exactamente el mismo árbol binario determinista'.

5. ¿Cómo se hizo el botón de regresión y qué lo controla?

Estrategia de Navegación: El botón de regreso o navegación retrospectiva se implementó con una doble estrategia técnica según el contexto de la vista:

- **1. Navegación Jerárquica en Obras:** En la Vista Detalle de Obra (resources/views/artworks/show.blade.php, L.18-34): Se implementó una barra de navegación contextual superior con el botón 'Back to Gallery'. El enlace apunta a route('home') y está acompañado por una ruta de migas de pan (Breadcrumbs) que permite regresar con precisión a la categoría específica de la obra (ej. Explore / Video / Título de la obra).
- **2. Botón Sensible al Contexto de Colección:** En los Tableros de Colección (resources/views/collections/show.blade.php, L.100-120): El botón de regreso es dinámico y condicional según el rol del visitante: si el usuario es el dueño de la colección (Auth::id() === $collection->user_id), el botón apunta a route('collections.index') con el texto 'Back to My Boards'; si el visitante es un tercero, apunta al perfil público del creador route('profile.public', $collection->user->slug) con el texto 'Back to [Nombre del Artista]'.
- **3. Control Reactivo en Modales:** En Modales y Visores Cinematográficos (viewer-modal.blade.php, slideshow-modal.blade.php): El control de regresión y cierre se gestiona mediante eventos de teclado en Alpine.js (@keydown.escape.window="close()") y clics fuera del modal (@click.outside="close()"), restaurando el foco visual y reactivando el scroll de la página de fondo.
- **4. Regresión en Formularios de Pasos:** En el Asistente de Onboarding (step2.blade.php, L.454): Se incluye el botón 'Back to Step 1' apuntando a route('onboarding.step1'), preservando los datos guardados en la base de datos.

6. ¿Cómo se hizo el control y manejo de las imágenes de Pinterest y qué archivos lo muestran?

Arquitectura de Pinterest: Pinterest es uno de los proveedores técnicamente más protegidos de la web. Su ingeniería de integración en Artfolio se compone de 4 fases:

- **Paso 1 — Desenrollado de URLs:** Resolución de URLs Cortas: Si el usuario ingresa un enlace acortado pin.it/..., el método fetchPinterestData() en EmbedService.php sigue las redirecciones HTTP mediante Guzzle y recupera la URL canónica definitiva ($res->effectiveUri()).
- **Paso 2 — Promoción a /originals/:** Extracción oEmbed y Escalado a Máxima Resolución: Consulta la API oEmbed oficial (https://www.pinterest.com/oembed.json?url=...). El JSON suele devolver una miniatura pequeña de 236 píxeles (/236x/). Nuestro algoritmo aplica una sustitución con expresiones regulares preg_replace('#/(236x|474x|564x|736x)/#', '/originals/', $thumb) y dispara una petición HEAD de 2 segundos para comprobar si el CDN de Pinterest tiene disponible la versión original sin compresión; si no existe, hace fallback a alta definición (/736x/).
- **Paso 3 — Optimización Grilla vs Visor:** Doble Resolución Adaptativa: En el modelo Artwork.php, el accessor getPreviewUrlAttribute() sirve la versión /474x/ en las tarjetas del explorador para ahorrar ancho de banda, mientras que getHighResUrlAttribute() sirve /originals/ cuando el usuario abre la obra en el visor cinematográfico o en la página individual.
- **Paso 4 — Desactivación de Referer:** Bypass Ético de Hotlink Protection: El CDN i.pinimg.com bloquea peticiones si detecta cabeceras 'Referer' de sitios desconocidos. Para solucionar esto sin descargar la imagen en nuestro disco, todas las etiquetas <img> en Blade incluyen el atributo referrerpolicy="no-referrer", haciendo que la imagen cargue limpia y nítida como un elemento nativo.

```
🗁 DIAGRAMA DE FLUJO — ALGORITMO DE EXTRACCIÓN Y ESCALADO DE PINTEREST
[URL pin.it/abc] --(Petición HTTP con Redirects)--> [URL canónica pinterest.com/pin/123]
       |
       v
[GET /oembed.json] --> [JSON devuelve: i.pinimg.com/236x/ab/cd/img.jpg]
       |
       v (Regex Regex Substitution: '/236x/' -> '/originals/')
[URL Candidata: i.pinimg.com/originals/ab/cd/img.jpg]
       |
       +--(HEAD Check 2s)--> ¿Responde 200 OK? --SI--> [Guarda en image_url: /originals/]
                                    |
                                    NO
                                    v
                  [Fallback Automático HD a: /736x/]
                                    |
                                    v
         [Blade <img referrerpolicy="no-referrer"> evade hotlinking de CDN]
```

POSIBLE REPREGUNTA DEL EVALUADOR: Si el profesor pregunta: '¿Esto viola derechos de autor de Pinterest?'. Respuesta: 'No, porque la imagen no se copia ni se aloja en nuestros servidores; se visualiza en streaming directo desde la CDN del proveedor mediante su protocolo oEmbed público, reconociendo el autor y redirigiendo tráfico orgánico al Pin original'.

```
🗁 ARCHIVOS DE CONTROL DE PINTEREST
-> app/Services/EmbedService.php (Líneas 147 a 250): Método fetchPinterestData().
-> app/Models/Artwork.php (Líneas 44 a 68): Accessors preview_url y high_res_url.
-> resources/views/components/artwork-card.blade.php: <img> con referrerpolicy="no-referrer".
-> resources/views/artworks/show.blade.php: Renderizado nativo en alta resolución.
```

7. ¿Cómo se hizo el control y manejo de las imágenes de DeviantArt y qué archivos lo muestran?

Mapeo Técnico: DeviantArt almacena ilustraciones de arte digital y concept art. Su integración opera a través de su backend oEmbed y un motor secundario de scraping de OpenGraph:

- **1. Petición oEmbed:** Consulta Oficial oEmbed: En app/Services/EmbedService.php (método fetchDeviantArtData(), L.256-315), el servidor realiza una petición GET a https://backend.deviantart.com/oembed?url=... con un User-Agent de navegador moderno.
- **2. Captura Full-Res:** Prioridad de Resolución Completa: El JSON devuelto por DeviantArt provee $data['url'] (que apunta al archivo de arte en tamaño completo) y $data['thumbnail_url']. Artfolio almacena preferentemente $data['url'], garantizando que los detalles de pinceladas y acabados digitales se aprecien en máxima fidelidad.
- **3. Créditos de Autoría:** Atribución Automática: Si el JSON contiene author_name, el sistema formatea el título como '[Título] - por [Autor]', reconociendo el crédito original.
- **4. Respaldo Ético:** Respaldo OpenGraph: Si una desviación tiene restricciones oEmbed, el backend descarga el HTML y extrae mediante expresiones regulares las etiquetas <meta property="og:image"> y <meta property="og:title">.

```
🗁 ARCHIVOS DE CONTROL DE DEVIANTART
-> app/Services/EmbedService.php (Líneas 256 a 315): fetchDeviantArtData().
-> resources/views/components/artwork-card.blade.php: Badge 'IMAGE' y renderizado.
-> resources/views/artworks/show.blade.php: Visualización en el lienzo principal.
```

8. ¿Cómo se hizo el control y manejo de los documentos de Google Docs y qué archivos lo muestran?

Lógica de Google Docs y Drive: Para que directores de narrativa, guionistas y game designers puedan exhibir guiones de cine, pitches o documentos de diseño (GDD) sin forzar al usuario a descargar archivos Word o PDF, Artfolio implementa un transformador bidireccional:

- **1. Extracción con Regex:** Detección y Parsing de ID: En app/Services/EmbedService.php (fetchGoogleDocsData(), L.490-560), la expresión regular #(?:docs|drive)\.google\.com/(document|presentation|spreadsheets|file)/d/([a-zA-Z0-9_-]+)#i captura el tipo de documento ($docType) y su identificador ($docId).
- **2. URL Limpia /preview:** Conversión a Modo Vista Limpia (/preview): Los enlaces copiados por los usuarios suelen terminar en /edit o ?usp=sharing. Artfolio los reescribe automáticamente a https://docs.google.com/{$docType}/d/{$docId}/preview (o drive.google.com/file/d/{$docId}/preview), eliminando barras de herramientas pesadas y creando un lector de solo lectura elegante.
- **3. Miniatura Oficial Drive:** Generación de Portada Visual sin API Key: Como Google Docs no tiene endpoint oEmbed público, Artfolio utiliza el generador oficial de miniaturas de Google Drive: https://drive.google.com/thumbnail?id={$docId}&sz=w800, generando una miniatura real de la primera página del documento.
- **4. Sandboxing:** Inyección en Iframe Seguro: Se genera un <iframe> con atributos de sandboxing sandbox="allow-scripts allow-same-origin allow-popups", permitiendo navegar las páginas del documento dentro de la plataforma.

```
🗁 ARCHIVOS DE CONTROL DE GOOGLE DOCS
-> app/Services/EmbedService.php (Líneas 490 a 560): fetchGoogleDocsData().
-> resources/views/components/artwork-card.blade.php: Badge 'DOC' y miniatura.
-> resources/views/components/viewer-modal.blade.php y artworks/show.blade.php: Iframe interactivo.
```

9. ¿Cómo se hizo el control y manejo de los modelos 3D de Sketchfab, cómo se cargan y qué archivos lo manejan?

Integración de Modelos 3D: Sketchfab permite renderizar modelos 3D interactivos con materiales PBR y luces en tiempo real utilizando la tecnología WebGL del navegador:

- **1. Protocolo oEmbed:** Consumo de oEmbed Oficial: En app/Services/EmbedService.php (fetchSketchfabData(), L.321-379), el backend consulta https://sketchfab.com/oembed?url=... obteniendo un JSON con el código HTML embebible oficial, título, nombre del artista y la miniatura de previsualización en alta definición.
- **2. Optimización WebGL:** Sanitización y Autoplay Controlado: El código iframe proporcionado por Sketchfab se procesa para garantizar parámetros óptimos de renderizado (autostart=0 para no saturar la GPU del cliente hasta que decida interactuar, ui_controls=1, ui_infos=0).
- **3. Atributos de Aceleración:** Permisos Especiales de Hardware: El iframe se inyecta con permisos para interactuar con la tarjeta gráfica del dispositivo: allow="autoplay; fullscreen; xr-spatial-tracking" y allowfullscreen="true".
- **4. Categorización:** Clasificación Automática: El sistema asigna render_type = 'iframe' y category = '3d', mostrando en la tarjeta el badge índigo '3D' con su icono de cubo tridimensional.

```
🗁 ARCHIVOS DE CONTROL DE SKETCHFAB 3D
-> app/Services/EmbedService.php (Líneas 321 a 379): fetchSketchfabData().
-> resources/views/components/artwork-card.blade.php: Badge '3D' y miniatura.
-> resources/views/components/viewer-modal.blade.php y artworks/show.blade.php: Iframe WebGL 3D.
```

10. ¿Cómo funciona el sistema de notificaciones y qué archivos lo controlan?

Arquitectura de Notificaciones: Artfolio implementa el sistema de notificaciones nativo de Laravel basado en base de datos (tabla 'notifications'):

- **1. Clases Específicas:** Clases Notification Creadas: (1) ArtworkLikedNotification (cuando un usuario aprecia una obra), (2) ArtworkCommentedNotification (cuando comentan en una obra) y (3) ArtworkBookmarkedNotification (cuando guardan la obra en favoritos). Todas heredan de Illuminate\Notifications\Notification e implementan el método via($notifiable) retornando ['database'].
- **2. Datos Persistidos:** Estructura del Payload JSON: En toDatabase($notifiable) o toArray(), cada notificación almacena: type, message, artwork_title, artwork_preview, actor_name, actor_avatar y url para redirigir directamente al contenido.
- **3. API de Notificaciones:** Controlador de Notificaciones (app/Http/Controllers/NotificationController.php): Provee el método index() que entrega vía JSON las 15 notificaciones más recientes formateadas con diffForHumans() (ej. 'hace 5 minutos') y el conteo unread_count, y el método markAsRead() para marcarlas individualmente o en lote.
- **4. Frontend Reactivo:** Componente en la Barra de Navegación: En resources/views/layouts/navigation.blade.php, un menú desplegable interactivo gestionado con Alpine.js consulta periódicamente o al hacer clic, muestra un badge rojo con el número de notificaciones no leídas y permite abrirlas sin recargar.

```
🗁 ARCHIVOS DEL SISTEMA DE NOTIFICACIONES
-> database/migrations/2026_11_04_051721_create_notifications_table.php: Tabla SQL 'notifications'.
-> app/Notifications/ArtworkLikedNotification.php, ArtworkCommentedNotification.php,
ArtworkBookmarkedNotification.php.
-> app/Http/Controllers/NotificationController.php (Líneas 1-73): Endpoint JSON y markAsRead().
-> resources/views/layouts/navigation.blade.php: Menú campana de notificaciones.
```

11. ¿Cómo funciona el sistema de tono claro y tono oscuro (Dark/Light Mode) y qué archivos lo manejan?

Implementación de Dark Mode: El sistema de Dark Mode está implementado bajo el estándar de Tailwind CSS basado en clases ('class strategy'):

- **1. Prevención de FOUC:** Script de Inicialización en el <head> (resources/views/layouts/app.blade.php, L.10-18): Para prevenir el molesto 'parpadeo de estilo no aplicado' (FOUC - Flash of Unstyled Content), se ejecuta un micro-script bloqueante sincrónico antes del renderizado del body: comprueba si en el almacenamiento local del navegador (localStorage.theme) existe 'dark' o si el sistema operativo del usuario tiene activo el modo oscuro (window.matchMedia('(prefers-color-scheme: dark)').matches). Si es verdadero, añade la clase 'dark' al elemento raíz <html>.
- **2. Toggle Persistente:** Conmutador Interactivo (Theme Toggle): Ubicado en la barra de navegación (resources/views/layouts/navigation.blade.php). Un botón con iconos de sol y luna permite al usuario alternar entre modos en cualquier momento. Al hacer clic, actualiza la clase en document.documentElement y persiste el valor 'dark' o 'light' en localStorage.
- **3. Variantes CSS:** Estilización Progresiva en Tailwind: Todos los componentes Blade emplean variantes dark:, por ejemplo: bg-white dark:bg-gray-900, text-gray-900 dark:text-gray-100 y border-gray-200 dark:border-gray-800, logrando un contraste cinematográfico accesible.

12. ¿Cómo funciona el sistema de sesiones, qué archivos lo manejan y cómo?

Arquitectura de Sesiones: El sistema de sesiones de Artfolio está diseñado para equilibrar la seguridad con la comodidad del usuario:

- **1. Persistencia y Tiempo:** Driver de Base de Datos y Duración: En .env y config/session.php está configurado SESSION_DRIVER=database y SESSION_LIFETIME=10 (10 minutos de inactividad). Las sesiones se almacenan en la tabla MySQL 'sessions', permitiendo rastrear la IP y el User-Agent del usuario activo.
- **2. Temporizador Reactivo:** Detección de Inactividad y Modal de Extensión: En resources/views/components/session-timeout-modal.blade.php se ejecuta un script que escucha eventos del usuario (mousemove, keydown, click, scroll). Si transcurren 8 minutos sin actividad (quedando 2 minutos para que la sesión expire), se despliega un modal con una cuenta regresiva de 120 segundos.
- **3. Endpoint Keep-Alive:** Extensión sin Recargar (Keep-Alive): Si el usuario pulsa 'Extender mi sesión', se dispara una petición POST asíncrona a la ruta /session/keep-alive (routes/web.php). El servidor regenera la marca de tiempo de la sesión y el modal se oculta sin que el usuario pierda lo que estaba escribiendo en sus formularios.

```
🗁 ARCHIVOS DE SESIÓN E INACTIVIDAD
-> config/session.php y .env: SESSION_DRIVER=database, SESSION_LIFETIME=10.
-> routes/web.php (Líneas 71 a 74): Route::post('/session/keep-alive', ...).
-> resources/views/components/session-timeout-modal.blade.php: Modal y lógica JS.
```

13. ¿Cómo funcionan los pasos de registro de un usuario nuevo y qué archivos los manejan?

Flujo de Registro en 3 Fases: El flujo de entrada de un usuario combina autenticación segura con un onboarding guiado progresivo:

- **1. Registro Breeze:** Fase 1 — Registro Inicial: El usuario ingresa nombre, email y contraseña en resources/views/auth/register.blade.php (procesado por RegisteredUserController.php). Se crea el registro básico con role = 'artist' y status = 'active'.
- **2. Verificación Obligatoria:** Fase 2 — Verificación de Correo: Laravel emite el evento Registered($user), disparando el email de verificación obligatoria con enlace firmado criptográficamente. Hasta confirmar el correo, el middleware 'verified' bloquea el acceso a funciones avanzadas.

- **3. Wizard de Onboarding:** Fase 3 — Onboarding Guiado en 3 Pasos (app/Http/Controllers/OnboardingController.php):
  • Paso 1 (step1.blade.php): Identidad artística (artistic_name), país de residencia mediante el selector inteligente con banderas conectado a /api/countries, y biografía breve.
  • Paso 2 (step2.blade.php): Perfil profesional: selección de software de dominio (Photoshop, Blender, etc.), habilidades artísticas y tipos de contratación (Freelance, Contrato, Comisiones).
  • Paso 3 (step3.blade.php): Encuadre visual: pega su avatar y banner, ajustando el recorte interactivo mediante arrastre y zoom.

```
🗁 ARCHIVOS DE REGISTRO Y ONBOARDING
-> app/Http/Controllers/Auth/RegisteredUserController.php: Registro base.
-> app/Http/Controllers/OnboardingController.php: Control del Wizard (step1, step2, step3).
-> resources/views/onboarding/step1.blade.php, step2.blade.php, step3.blade.php.
-> database/migrations/*_add_onboarding_and_professional_fields_to_users_table.php.
```

14. ¿Cómo se optimizó la página? ¿De qué manera y en qué archivos?

Estrategia Integral de Performance: La optimización de Artfolio aborda el rendimiento tanto en el servidor (backend) como en el navegador (frontend):

- **1. Memoización Backend:** Eliminación de Consultas N+1 con once(): En resources/views/components/artwork-card.blade.php, para verificar si el usuario autenticado dio like o guardó en favoritos cada obra de la grilla, se utiliza la función helper once() de Laravel 11/12. Esto ejecuta una sola consulta por petición HTTP en lugar de consultar la base de datos 50 veces para 50 tarjetas.
- **2. Indexación B-Tree:** Índices de Base de Datos Compuestos: En database/migrations/2026_10_12_020000_add_status_category_created_at_index_to_artworks_table.php, se indexó conjuntamente ['status', 'category', 'created_at']. Las consultas con filtrado y ordenamiento se resuelven directamente en el árbol de índices sin escaneo de tablas.
- **3. Caché de Endpoints:** Caché de Larga Duración en API de Países: El endpoint /api/countries utiliza Cache::remember('global_countries_list_v1', 86400 * 30, ...) con un tiempo de vida de 30 días, respondiendo en menos de 2 milisegundos.
- **4. Lazy Loading Nativo:** Carga Diferida de Imágenes (Lazy Loading): Todas las etiquetas <img> en las tarjetas utilizan loading="lazy" y decoding="async", asegurando que el navegador solo descargue las imágenes que están visibles en el viewport.
- **5. Bundling con Vite:** Compilación y Minificación con Vite: Los estilos Tailwind y scripts Alpine.js se compilan en un único bundle CSS/JS minificado y con fingerprinting de caché (app-Cb5G8TmM.css y app-0FAcbh4c.js), comprimidos con Gzip/Brotli.

15. ¿Cómo se implementó el uso de Mailpit y por qué se eligió Mailpit?

Implementación de Mailpit: Mailpit es un servidor de pruebas SMTP ultraliviano y herramienta de inspección de correos para desarrolladores:

- **1. Configuración Local:** Configuración en .env: Se define MAIL_MAILER=smtp, MAIL_HOST=127.0.0.1 y MAIL_PORT=1025. Cuando la aplicación envía un correo (confirmación de cuenta, alerta de enlace caído o contacto con artista), Laravel lo transmite vía socket SMTP local a Mailpit en menos de 5 milisegundos.
- **2. Acceso en Vivo:** Bandeja Web Interactiva en 127.0.0.1:8025: Mailpit ofrece una interfaz web donde se ven llegar los correos en tiempo real, permitiendo previsualizar el diseño HTML responsivo, verificar cabeceras MIME y hacer clic en el botón de confirmación de cuenta durante la demostración.
- **3. Justificación:** ¿Por qué se eligió?: Porque elimina por completo el riesgo de enviar correos accidentales a direcciones reales durante pruebas, no requiere crear cuentas falsas de Gmail ni lidiar con bloqueos de puertos o autenticación en dos pasos, y funciona 100% offline.

16. ¿Cómo se creó el sistema de colecciones, qué archivos lo manejan y cómo?

Arquitectura de Colecciones: El módulo de Colecciones permite a los usuarios agrupar obras en tableros temáticos (Moodboards):

- **1. Modelo y Estructura SQL:** Modelo y Tablas (Collection.php, tabla 'collections' y tabla pivote 'artwork_collection'): Cada colección tiene user_id, name, description, visibility ('public', 'unlisted', 'private'), share_token (token criptográfico de 32 caracteres), preview_artwork_ids (JSON con IDs de obras destacadas para la portada mosaico) y sort_order.
- **2. Política de Privacidad:** Autorización Granular (app/Policies/CollectionPolicy.php): (a) 'public' es visible en el perfil del creador; (b) 'unlisted' se oculta del perfil pero es visible para cualquiera con el enlace ?token={share_token}; (c) 'private' solo puede ser vista por su dueño.
- **3. Componente Quick-Create:** Selector Rápido en Tarjetas (x-collection-picker.blade.php): Un dropdown AJAX en cada obra que lista las colecciones del usuario, permite añadir/quitar con toggle inmediato y ofrece la opción 'Create collection' que crea la colección y agrega la obra en una sola acción.
- **4. Presentación Slideshow:** Modo Slideshow a Pantalla Completa: En collections/show.blade.php se incluye el visor x-slideshow-modal.blade.php para proyectar las piezas de la colección una a una sin distracciones.

17. ¿Cómo se creó el sistema de comentarios? ¿Cómo funciona? ¿Se guardan en la base de datos?

Respuesta y Funcionamiento: Sí, los comentarios se guardan de forma permanente y normalizada en la base de datos MySQL en la tabla 'artwork_comments':

- **1. Persistencia en Base de Datos:** Estructura SQL: La tabla 'artwork_comments' almacena: id, artwork_id (FK con borrado en cascada), user_id (FK al autor del comentario), content (texto de hasta 1000 caracteres sanitizado) y marcas de tiempo created_at/updated_at.
- **2. Lógica y Moderación Dual:** Controlador Dedicado (app/Http/Controllers/ArtworkCommentController.php): Provee el método store() que valida el comentario, previene envíos vacíos y notifica al dueño de la obra mediante ArtworkCommentedNotification. El método destroy() implementa una política de moderación doble: pueden borrar el comentario tanto el usuario que lo redactó como el artista dueño de la obra donde se comentó.
- **3. Visualización en Blade:** Renderizado en la Vista Detalle: En resources/views/artworks/show.blade.php se muestra el hilo de comentarios con avatar, nombre, insignia de verificación y tiempo transcurrido, permitiendo enriquecer la retroalimentación artística.

18. ¿Cómo se implementó el acceso restringido a la página de administración?

Blindaje Administrativo: La seguridad del panel de administración (/admin) se blindó mediante un middleware específico de autorización:

- **1. Middleware de Inspección:** Middleware EnsureIsAdmin (app/Http/Middleware/EnsureIsAdmin.php): Se ejecuta antes de cualquier ruta bajo el prefijo 'admin'. Verifica: if (!$request->user() || !$request->user()->isAdmin()) { abort(403, 'Acceso restringido. Solo administradores.'); }.
- **2. Verificación de Rol:** Método en el Modelo User: En app/Models/User.php se define public function isAdmin(): bool { return $this->role === 'admin'; }.
- **3. Enrutamiento Protegido:** Agrupación en Rutas: En routes/web.php (L.172-220), todas las rutas administrativas están envueltas en Route::prefix('admin')->name('admin.')->middleware(['auth', EnsureIsAdmin::class]), impidiendo que usuarios comunes o visitantes accedan a auditoría, usuarios, reportes o taxonomía.

19. ¿Cómo funcionan los modos de búsqueda? ¿Qué archivos lo manejan y cómo?

Motores de Búsqueda: Artfolio cuenta con dos motores de búsqueda complementarios:

- **1. Explorador de Obras:** Buscador Facetado de Obras (ExplorerController.php y explorer/index.blade.php): Permite buscar por término clave en títulos, descripciones y etiquetas. Incluye filtros por tipo de medio (imagen, video, 3D, audio, doc), software utilizado (Photoshop, Blender, etc.) y ordenamiento multidimensional (recientes, populares, más vistas, más apreciadas).
- **2. Directorio de Artistas:** Directorio Global de Artistas (ArtistExplorerController.php y artists/index.blade.php): Diseñado para reclutadores de la industria. Permite buscar por nombre y especialidad, y filtrar por: país de residencia (conectado a /api/countries), software de dominio, habilidades técnicas, disponibilidad para comisiones/contratación y artistas verificados.

20. ¿Cómo se hizo el display y muestreado de las obras? ¿Por qué están organizadas así?

Estructura Visual Simétrica: El diseño de las tarjetas de obra (resources/views/components/artwork-card.blade.php) fue estructurado bajo un CSS Grid de 3 columnas (grid-cols-[1fr_auto_auto] items-center gap-x-2 gap-y-2) para garantizar simetría visual absoluta:

- **1. Fila de Título y Aprecio:** Fila 1 (En línea con el Título): A la izquierda el título con recorte elegante (truncate); en la columna 2 el botón de Like con su contador; en la columna 3 el botón de Favoritos.
- **2. Fila de Autor y Colección:** Fila 2 (En línea con el Artista): A la izquierda el avatar del creador, su nombre y la insignia de verificación; en la columna 2 el contador de Vistas; en la columna 3 el selector de Colecciones.
- **3. Simetría de Columnas:** Alineación Vertical Perfecta: El corazón de likes queda exactamente encima del ojo de vistas (ambos con iconos w-4 h-4 y tipografía mono), y el marcador de favoritos queda exactamente encima del selector de colecciones (con size='sm' w-4 h-4).

21. ¿Qué es Tailwind? ¿Qué usos se le hicieron? ¿Por qué?

Definición y Justificación: Tailwind CSS es un framework CSS utility-first que permite construir interfaces complejas componiendo clases de utilidad atómicas directamente en el HTML/Blade:

- **1. Rendimiento y Mantenibilidad:** ¿Por qué se eligió?: Elimina los archivos CSS gigantescos y difíciles de mantener. Con Tailwind, el diseño es 100% responsivo mediante prefijos de pantalla (sm:, md:, lg:, xl:), tiene soporte nativo para modo oscuro con la variante dark:, y durante la compilación con Vite, el compilador purga todas las clases no utilizadas, produciendo una hoja de estilo minificada de solo ~120 KB.
- **2. Aplicación en el Proyecto:** Usos Clave en Artfolio: Grid responsivo de tarjetas, modales cinemáticos con desenfoque de fondo (backdrop-blur-md), paleta índigo estandarizada y transiciones de aceleración por hardware.

22. ¿Qué se añadió en el .env que no es parte de Laravel por defecto? ¿Por qué?

Variables Específicas de Artfolio: En el archivo .env se incorporaron variables de entorno personalizadas para desacoplar la configuración de la infraestructura:

- **1. Configuración de Cloudflare:** TURNSTILE_ENABLED, TURNSTILE_SITE_KEY, TURNSTILE_SECRET_KEY: Permiten habilitar o deshabilitar la protección anti-bot de Cloudflare Turnstile en entornos de desarrollo y producción sin modificar una sola línea de código PHP.
- **2. Credenciales del Administrador:** ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD: Utilizadas por DatabaseSeeder.php para inicializar la cuenta del administrador con credenciales seguras y personalizables.
- **3. Tiempo de Sesión Controlado:** SESSION_LIFETIME=10: Reducido a 10 minutos para probar y activar el modal de advertencia de inactividad de sesión.

23. ¿Dónde y cómo se maneja la whitelist? ¿En qué archivo?

Ubicación y Mecanismo: La lista blanca (Whitelist) y el blindaje Anti-SSRF se gestionan de forma centralizada en el archivo app/Services/EmbedService.php:

- **1. Lista de Dominios:** Propiedad $allowedDomains (L.12-32): Diccionario asociativo con los dominios permitidos por proveedor: 'pinterest' (pinterest.com, pin.it), 'deviantart' (deviantart.com, sta.sh), 'youtube' (youtube.com, youtu.be), 'sketchfab' (sketchfab.com), 'spotify' (spotify.com), 'googledocs' (docs.google.com, drive.google.com).
- **2. Bloqueo Anti-SSRF:** Método processUrl() (L.41-90): Bloquea activamente direcciones IP crudas y 'localhost'. Limpia el subdominio 'www.' y valida que el host termine estrictamente en uno de los dominios autorizados; de lo contrario, aborta la petición con una excepción descriptiva.

24. ¿Cómo funciona la vista en móvil? ¿Hay algún archivo que maneje eso?

Adaptabilidad Móvil: La adaptabilidad móvil no depende de un archivo separado, sino de un diseño responsivo integrado ('Mobile-First') con Tailwind CSS y Alpine.js:

- **1. Drawer de Navegación:** Menú de Navegación Responsivo: En resources/views/layouts/navigation.blade.php, en pantallas móviles (<640px) la barra superior muestra un botón hamburguesa interactivo con Alpine.js (x-show="open") que despliega un menú vertical táctil con buscador, notificaciones y perfil.
- **2. Rejilla Fluida:** Grilla Adaptativa: Las obras utilizan grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4, ajustando fluidamente la cantidad de columnas según el ancho del dispositivo sin romper proporciones.
- **3. Diálogos Táctiles:** Modales Full-Width: Los modales se configuran con max-w-[calc(100vw-2rem)] para que en celulares ocupen el ancho completo de la pantalla sin desbordamiento horizontal.

25. ¿Qué problemas surgieron a la hora de hacer la página? ¿Cómo se resolvieron?

- **1. Hotlinking de Pinterest:** Problema: Los servidores de Pinterest (i.pinimg.com) devolvían error 403 Forbidden al intentar mostrar las imágenes en nuestra web local. Solución: Se investigó la política de protección contra hotlink y se añadió el atributo referrerpolicy="no-referrer" en todas las etiquetas <img>, logrando que el navegador omita el origen y cargue la imagen limpiamente.
- **2. Miniaturas de YouTube 404:** Problema: Enlaces cortos de YouTube (youtu.be/...) o enlaces largos no mostraban la miniatura HD maxresdefault.jpg (daba 404). Solución: Se programó una verificación con petición HTTP HEAD; si maxresdefault responde 200 la conserva, y si devuelve 404 hace degradación elegante a hqdefault.jpg.
- **3. Imágenes Pixeladas:** Problema: Pinterest oEmbed entrega miniaturas de 236px que se veían pixeladas. Solución: Se construyó un sustituidor con expresiones regulares que transforma la ruta a /originals/ y comprueba su existencia en tiempo real antes de guardar.
- **4. Cuello de Botella N+1:** Problema: Al renderizar 50 tarjetas de obras en el explorador, la aplicación ejecutaba 100 consultas SQL adicionales para comprobar si cada obra tenía like o favorito del usuario (problema N+1). Solución: Se aplicó memoización mediante el helper once() en la tarjeta, reduciendo las consultas a 2 por petición.

BLOQUE 2 — PREGUNTAS DE ARQUITECTURA AVANZADA, SEGURIDAD Y GOBERNANZA

Interrogatorio Técnico Senior: Auditoría, Taxonomía, Concurrencia y Tests

10 preguntas exhaustivas diseñadas para defender la robustez de Artfolio ante evaluadores rigurosos y justificar la arquitectura ante tribunales de evaluación.

26. ¿Cómo se previene el ataque de CSRF (Cross-Site Request Forgery) en las peticiones AJAX de likes y colecciones?

Protección Anti-CSRF: En toda aplicación con acciones interactivas asíncronas, un sitio malicioso podría intentar enviar peticiones en nombre del usuario autenticado si no existiera una validación de token de sesión:

- **1. Token en el DOM:** Etiqueta Meta CSRF en el Layout: En resources/views/layouts/app.blade.php se inyecta <meta name="csrf-token" content="{{ csrf_token() }}">, que genera un token criptográfico único por sesión.
- **2. Cabecera HTTP X-CSRF-TOKEN:** Inyección en Cabeceras Fetch: Cada llamada asíncrona de Alpine.js en artwork-card.blade.php y collection-picker.blade.php incluye: headers: { 'X-CSRF-TOKEN': document.querySelector('meta[name=csrf-token]').content, 'Accept': 'application/json' }.
- **3. Verificación en Servidor:** Validación por Middleware: El middleware VerifyCsrfToken intercepta la petición y valida la firma criptográfica antes de permitir la ejecución del controlador.

27. ¿Cómo se implementó el sistema de Likes y por qué no se usa un simple incremento numérico directo en la base de datos?

Diseño de la Tabla artwork_likes: Incrementar una columna likes = likes + 1 directamente en la tabla de obras es un grave error de diseño porque no previene el fraude ni permite saber quién dio like:

- **1. Restricción a Nivel SQL:** Tabla Pivote con Índice Único Compuesto: En database/migrations/2026_10_23_040000_create_artwork_likes_and_fields_table.php, la tabla 'artwork_likes' almacena user_id y artwork_id con una restricción única $table->unique(['user_id', 'artwork_id']). Esto impide físicamente que un usuario dé like dos veces a nivel de motor de base de datos.
- **2. Desnormalización Atómica:** Columna Desnormalizada likes_count para Rendimiento: Para no ejecutar un COUNT(*) en cada consulta del explorador, se mantiene la columna likes_count en 'artworks', que se incrementa o decrementa dentro de una transacción atómica cuando se inserta o elimina el registro en 'artwork_likes'.
- **3. Evento Alpine.js:** Sincronización Reactiva en Tiempo Real: Al dar like, el frontend emite un CustomEvent 'artwork-like-updated' en window con el ID de la obra y el nuevo conteo, sincronizando automáticamente todas las tarjetas que muestren esa misma obra en la pantalla sin recargar.

28. ¿Cómo se calculan las Visitas Únicas y cómo se previenen conteos inflados artificialmente?

Mecanismo Anti-Inflación: Para que la métrica de visitas sea fidedigna ante clientes y reclutadores, ArtworkViewController.php implementa una política de retención por cookies:

- **1. Cookie de Navegación:** Cookie de Sesión: Cuando un visitante abre la vista /artworks/{id}, el controlador comprueba si existe la cookie 'viewed_artwork_' . $id. Si la cookie ya está presente, no incrementa el contador.
- **2. Incremento Protegido:** Persistencia Condicional: Solo si es una visita nueva se ejecuta $artwork->increment('views_count') y se adosa la cookie a la respuesta HTTP con una expiración de 24 horas.

29. ¿Cómo funciona el Módulo de Auditoría del Sistema (/admin/audit-logs) y qué datos forenses captura?

Trazabilidad Empresarial: El módulo de auditoría garantiza cumplimiento normativo y trazabilidad administrativa inmutable:

- **1. Servicio Centralizado:** Servicio AuditLogger (app/Services/AuditLogger.php): Expone métodos estáticos como AuditLogger::log(event, description, auditable, oldValues, newValues, severity).
- **2. Payload Forense:** Datos Capturados: Cada registro en la tabla 'audit_logs' almacena: ID del usuario que ejecutó la acción, evento (ej. 'user.suspended', 'artwork.deleted', 'report.resolved'), modelo afectado, diferencial JSON de valores antes y después, IP remota, User-Agent y nivel de severidad ('info', 'warning', 'danger').
- **3. Interfaz de Monitoreo:** Panel Administrativo: En resources/views/admin/audit_logs/index.blade.php los administradores pueden filtrar por severidad, buscar por término y exportar los registros para auditorías de seguridad.

30. ¿Cómo funciona el Módulo de Taxonomía Curada y por qué se separó de los tags libres?

Control de Calidad de Datos: Permitir que los usuarios ingresen texto completamente libre para habilidades y software genera duplicados caóticos (ej. 'photoshop', 'PhotoShop', 'PS', 'adobe photoshop'):

- **1. Diccionario Oficial:** Tabla 'curated_tags' y Modelo CuratedTag.php: Administra un diccionario oficial categorizado en: 'software' (Blender, Photoshop, Unreal Engine, ZBrush), 'skills' (Concept Art, 3D Modeling, Digital Painting) y 'styles'.
- **2. Gestión Centralizada:** Seeder y Panel Admin: TaxonomySeeder.php inicializa el catálogo, y en /admin/skills-software los administradores pueden añadir, editar, eliminar o marcar tags como 'sugeridos' para que aparezcan destacados en el Onboarding y en el Explorador.

31. ¿Cómo se maneja el contenido sensible o maduro (+18) para cumplir con normativas de protección de menores?

Protección de Audiencias: Para operar conforme a las políticas de plataformas globales y proteger a audiencias sensibles:

- **1. Declaración por Obra:** Bandera is_mature en 'artworks': Permite al creador declarar con transparencia si su obra contiene temática para mayores de edad.
- **2. Desenfoque Preventivo:** Preferencia de Usuario wantsMatureBlurred(): En el modelo User.php se consulta si el usuario desea desenfocar obras maduras. Para visitantes no autenticados, el desenfoque está activado por defecto.
- **3. Interfaz de Revelación Segura:** Desenfoque CSS blur-lg en Tarjetas: Las imágenes se cargan con filtro blur-lg y una advertencia superpuesta, permitiendo al usuario desenfocarla temporalmente mediante un clic consciente.

32. ¿Cómo se garantiza la seguridad en la inyección de iframes de terceros (YouTube, Sketchfab, Google Docs)?

Aislamiento de Iframes: Inyectar contenido HTML de sitios externos representa un riesgo de XSS si no se aísla adecuadamente el contexto de ejecución:

- **1. Sandboxing:** Atributo Sandbox Estricto: Todos los iframes inyectados en viewer-modal.blade.php y artworks/show.blade.php incluyen la directiva sandbox="allow-scripts allow-same-origin allow-popups allow-presentation". Esto impide que el iframe acceda a las cookies de sesión de Artfolio o ejecute redirecciones maliciosas.
- **2. Sanitización en Servidor:** Sanitización en EmbedService: El HTML del iframe se construye en el servidor a partir de IDs validados con expresiones regulares, nunca a partir de código crudo proporcionado por el usuario.

33. ¿Cómo funciona el sistema de SoftDeletes y el comando de purga automática de 30 días?

- **1. Papelera de Reciclaje:** Borrado Lógico con SoftDeletes: Al eliminar una obra, Laravel llena la columna deleted_at sin borrar el registro físico. Las consultas normales la excluyen automáticamente mediante el scope global SoftDeletingScope.

- **2. Purga Automática de 30 Días:** Comando Programado artworks:purge-trash: En routes/console.php se programó el comando que consulta Artwork::onlyTrashed()->where('deleted_at', '<=', now()->subDays(30))->forceDelete(), purgando definitivamente obras que superen el periodo de gracia de 30 días.

34. ¿Cómo funciona el Bloqueo Bidireccional entre usuarios en el explorador y perfil?

- **1. Invisibilidad Mutua:** Tabla 'user_blocks' y Exclusión Mutua: Almacena blocker_id y blocked_id. Si el Usuario A bloquea al Usuario B, en ExplorerController y ArtistExplorerController se calcula la unión de ambos IDs y se excluyen todas sus obras de la grilla ($query->whereNotIn('user_id', $excludedIds)).
- **2. Bloqueo de Acceso Directo:** Bloqueo en Perfil Público: Si el usuario bloqueado intenta visitar la URL del perfil del bloqueador, se aborta la petición con un error HTTP 404/403.

35. ¿Cómo se diseñó la arquitectura de pruebas automatizadas (173 tests en PHPUnit)?

- **1. Alcance de la Suite:** Cobertura Integral de 173 Tests (816 aserciones): Distribuidos en pruebas unitarias y de integración en tests/Feature/ que cubren: autenticación, verificación de correo, turnstile, CRUD de obras, whitelists anti-SSRF, extractor de Pinterest/DeviantArt/YouTube/Docs/Sketchfab, colecciones y privacidad, papelera soft-deletes, contenido sensible (+18), comentarios, likes, vistas únicas y panel administrativo.
- **2. Determinismo:** Aislamiento con RefreshDatabase: Cada test se ejecuta en una base de datos limpia con transacciones que se revierten al finalizar, garantizando resultados deterministas y reproducibles.

BLOQUE 3 — MATRIZ RESUMEN DE ARCHIVOS Y GUÍA RÁPIDA DE CONSULTA

Cheat Sheet Maestro: Localización Inmediata para el Examen Oral

Tabla de referencia exhaustiva para responder de manera instantánea ante cualquier consulta del evaluador sobre archivos y métodos del proyecto.

Módulo del Proyecto	Archivos Principales Involucrados	Métodos, Líneas o Características Clave
Cloudflare Turnstile	app/Rules/TurnstileRule.php resources/views/auth/login.blade.php	validate() contra challenges.cloudflare.com Div .cf-turnstile en login
Whitelists y Anti-SSRF	app/Services/EmbedService.php	$allowedDomains (L.12-32), processUrl() (L.41-90)
Integración Pinterest	app/Services/EmbedService.php app/Models/Artwork.php	fetchPinterestData(), regex /originals/ referrerpolicy="no-referrer"
Integración DeviantArt	app/Services/EmbedService.php	fetchDeviantArtData(), backend.deviantart.com/oembed
Integración Google Docs	app/Services/EmbedService.php	fetchGoogleDocsData(), endpoint /preview y miniatura Drive
Integración Sketchfab 3D	app/Services/EmbedService.php	fetchSketchfabData(), iframe WebGL interactivo sandbox
Integración YouTube	app/Services/EmbedService.php app/Models/Artwork.php	fetchYouTubeData(), fallback maxresdefault a hqdefault
Sistema de Notificaciones	app/Http/Controllers/NotificationController.php app/Notifications/*.php	index() JSON, markAsRead(), tabla notifications
Modo Oscuro / Claro	resources/views/layouts/app.blade.php resources/views/layouts/navigation.blade.php	Script anti-FOUC en <head>, localStorage.theme Clases 'dark:' de Tailwind
Sesión y Keep-Alive	config/session.php resources/views/components/session-timeout-modal.blade.php	SESSION_LIFETIME=10, modal a los 8 min POST /session/keep-alive
Onboarding y Registro	app/Http/Controllers/OnboardingController.php resources/views/onboarding/*.blade.php	Wizard step1, step2, step3 con drag & zoom
API de Países con Banderas	routes/web.php (L.15-57) resources/views/onboarding/step1.blade.php	GET /api/countries con Cache 30d y Triple Fallback
Directorio de Artistas	app/Http/Controllers/ArtistExplorerController.php resources/views/artists/index.blade.php	Filtros por país, comisiones, software y verificación
Buscador Facetado de Obras	app/Http/Controllers/ExplorerController.php resources/views/explorer/index.blade.php	Filtros por medio, software, tags y ordenamiento múltiple
Vista Detalle de Obra	resources/views/artworks/show.blade.php app/Http/Controllers/ArtworkController.php	Experiencia Pinterest, likes AJAX, comentarios y vistas
Sistema de Likes	app/Http/Controllers/ArtworkLikeController.php	Tabla artwork_likes con índice único

	database/migrations/*_artwork_likes_*.php	compuesto CustomEvent 'artwork-like-updated'
Sistema de Comentarios	app/Http/Controllers/ArtworkCommentController.php app/Models/Comment.php	Tabla artwork_comments, moderación dual autor/dueño
Tableros de Colecciones	app/Http/Controllers/CollectionController.php resources/views/collections/show.blade.php	preview_artwork_ids (portada mosaico) x-slideshow-modal a pantalla completa
Selector de Colección	resources/views/components/collection-picker.blade.php	Dropdown AJAX rápido con size='sm' y quick-create
Tarjetas Simétricas Obra	resources/views/components/artwork-card.blade.php	CSS Grid 3 columnas (grid-cols-[1fr_auto_auto]) Alineación título-likes y artista-vistas
Contenido Maduro (+18)	app/Models/Artwork.php app/Models/User.php	is_mature, wantsMatureBlurred(), desenfoque blur-lg
Auditoría del Sistema	app/Services/AuditLogger.php app/Http/Controllers/Admin/AuditLogController.php	Tabla audit_logs con severidad y diferencial JSON
Taxonomía Curada	app/Models/CuratedTag.php app/Http/Controllers/Admin/TaxonomyController.php	Catálogo oficial skills/software, TaxonomySeeder.php
Panel de Administración	app/Http/Middleware/EnsureIsAdmin.php routes/web.php (L.172-220)	Middleware EnsureIsAdmin (abort 403) Rutas agrupadas con prefijo 'admin'
Papelera SoftDeletes	app/Http/Controllers/TrashController.php routes/console.php	Trait SoftDeletes, purga 'artworks:purge-trash' (30d)
Bloqueo entre Usuarios	app/Http/Controllers/UserBlockController.php table user_blocks	Exclusión mutua whereNotIn('user_id', $excludedIds)
Seguridad de Cabeceras	app/Http/Middleware/SecurityHeadersMiddleware.php	CSP, X-Frame-Options, X-Content-Type-Options
Suite de Pruebas (173)	tests/Feature/*.php phpunit.xml	173 pruebas automáticas con 816 aserciones al 100%

BLOQUE 4 — SIMULACRO DE DEMOSTRACIÓN EN VIVO Y COMANDOS DE DIAGNÓSTICO

10 Escenarios Prácticos de Evaluación: Terminal, Debugging y Demostración

Protocolo paso a paso para responder con seguridad cuando el evaluador solicite demostraciones prácticas, ejecución de comandos artesanales o inspección de datos en caliente.

Escenario 1: 'Reinicia la base de datos y vuelve a poblarla con datos de prueba'

Comando y Explicación: Si el profesor solicita restablecer la aplicación a su estado inicial impecable con todas las obras, usuarios de prueba, taxonomía y registros de auditoría:

```
COMANDO CLI ARTISAN
php artisan migrate:fresh --seed
```

Lo que debes explicar verbalmente: Este comando elimina todas las tablas de MySQL, vuelve a ejecutar las 17 migraciones en su estricto orden cronológico e invoca DatabaseSeeder.php, que dispara sucesivamente: AdminSeeder (crea al administrador), TaxonomySeeder (puebla los tags oficiales curados), DemoContentSeeder y RichCommunitySeeder (crea artistas con diversas redes sociales, obras con enlaces 100% reales de Pinterest/DeviantArt/Sketchfab, comentarios, likes, logs de auditoría y reportes).

Escenario 2: 'Demuestra que el servidor de correo local está funcionando'

Comando CLI Artesanal: Para comprobar la conectividad del protocolo SMTP sin necesidad de registrar un usuario en el navegador:

```
COMANDO ARTESANAL DE DIAGNÓSTICO DE CORREO
php artisan mail:test profesor@ejemplo.com
```

Demostración Visual: Inmediatamente abre en el navegador http://127.0.0.1:8025 (la bandeja web de Mailpit). El profesor verá aparecer el correo con asunto 'Artfolio Test Email', demostrando que el socket local en el puerto 1025 interceptó el correo en milisegundos con sus cabeceras y diseño HTML completo.

Escenario 3: 'Ejecuta la suite de pruebas automatizadas y muestra que todo pasa'

Comando de Testing: Si el evaluador pide verificar la calidad del software mediante testing automatizado:

```
EJECUCIÓN DE SUITE PHPUNIT
php artisan test
```

Defensa Verbal: La consola reportará: 'Tests: 173 passed (816 assertions)'. Explica que la suite valida el 100% de los flujos críticos: autenticación, verificación de email, extractor de proveedores, seguridad de whitelists, permisos de moderación, papelera, likes y contenido maduro.

Escenario 4: 'Muestra cómo el sistema detecta y alerta sobre un enlace caído'

Comando Programable de Salud: Para demostrar la capacidad de auto-recuperación y salud del catálogo (Self-Healing):

```
COMANDO DE VERIFICACIÓN AUTOMÁTICA DE ENLACES
php artisan artworks:check-links
```

Explicación Técnica: El comando recorre las obras activas enviando peticiones HEAD no invasivas. Si detecta un código HTTP 404 o 410, marca link_status = 'down', oculta automáticamente la obra del explorador público para proteger la experiencia del visitante y despacha el correo transaccional LinkDownAlert al artista con el enlace para actualizar su publicación.

Escenario 5: 'Demuestra cómo se limpia la papelera de obras con más de 30 días'

Comando de Purga: Para evidenciar la automatización del ciclo de vida de los datos eliminados:

```
🗁 COMANDO DE PURGA AUTOMÁTICA DE SOFTDELETES
php artisan artworks:purge-trash
```

Lógica de Retención: Explica que este comando consulta Artwork::onlyTrashed()->where('deleted_at', '<=', now()->subDays(30))->forceDelete(). Si una obra lleva menos de 30 días, permanece en la papelera permitiendo su restauración por el artista; si supera los 30 días, se purga definitivamente de la base de datos liberando espacio.

Escenario 6: 'Interactúa con los modelos directamente en la consola (Tinker)'

Sesión Interactiva con Tinker: Si el profesor solicita inspeccionar un usuario o cambiar un rol sin usar la interfaz gráfica:

```
🗁 CONSOLA REPL LARAVEL TINKER
php artisan tinker
> $user = App\Models\User::where('email', 'admin@artfolio.test')->first();
> $user->role;
// Devuelve 'admin'
> $user->is_verified;
// Devuelve true
> App\Models\Artwork::count();
// Devuelve el total de obras activas en MySQL
```

Escenario 7: 'Demuestra la prevención de ataques SSRF intentando registrar 127.0.0.1'

Prueba de Seguridad en Vivo: Si el evaluador cuestiona la efectividad de la lista blanca:

Demostración y Código: Entra al formulario de 'Publicar Obra' e intenta ingresar http://127.0.0.1:8000 o http://localhost/secreto. Al pulsar 'Previsualizar', la interfaz responderá de inmediato con el mensaje de error: 'URL no permitida. Solo se aceptan enlaces de proveedores autorizados (Pinterest, DeviantArt, YouTube, Sketchfab, Google Docs, Spotify)'. Muestra en pantalla app/Services/EmbedService.php líneas 68-70 para enseñar el bloqueo explícito filter_var($host, FILTER_VALIDATE_IP) || in_array($cleanHost, ['localhost']).

Escenario 8: 'Muestra dónde se registran los eventos de auditoría administrativa'

Inspección de Auditoría Forense: Inicia sesión como administrador e ingresa a la ruta /admin/audit-logs:

Defensa de Gobernanza: Muestra cómo cada inicio de sesión, cambio de rol o eliminación de obra queda registrado con: marca de tiempo exacta, ID del usuario responsable, dirección IP del cliente, nivel de severidad (badge verde 'info', amarillo 'warning' o rojo 'danger') y el detalle en formato JSON con los valores anteriores y nuevos. Explica que esto garantiza trazabilidad inalterable ante auditorías corporativas.

Escenario 9: 'Demuestra el funcionamiento del desenfoque preventivo para obras +18'

Demostración de Contenido Maduro: Abre una ventana de incógnito (modo visitante anónimo) y navega por el explorador:

Demostración Visual: Localiza una obra marcada con is_mature = true. El evaluador verá que la miniatura se carga con un desenfoque gaussiano severo (blur-lg) y una tarjeta de advertencia 'Contenido Maduro (+18)'. Al pulsar el botón 'Mostrar contenido', la imagen se revela limpiamente mediante Alpine.js sin recargar la página. Explica que esto cumple estrictamente con normativas internacionales de protección de menores.

Escenario 10: 'Muestra la sincronización en tiempo real del contador de Likes'

Demostración de Concurrencia y Reactividad: En una obra visible en el explorador, haz clic en el botón de corazón de apreciación:

Defensa de Arquitectura Frontend: El contador de likes se incrementa al instante de forma asíncrona (petición POST /artworks/{id}/like) cambiando el icono a color rosado activo. Explica que el evento Alpine.js 'artwork-like-updated' se propaga por el bus de eventos global de la ventana (window.dispatchEvent), garantizando que si esa misma obra aparece en un carrusel o en otra sección de la página, sus contadores se actualizan simultáneamente.